# Nested Classes

The Java programming language allows you to define a class within another class. Such a class is called a nested class and is illustrated here:

```
class OuterClass {
    ...
    class NestedClass {
        ...
    }
}
```

Nested classes are divided into two categories: **non-static** and **static**. Non-static nested classes are called inner classes. Nested classes that are declared static are called static nested classes.

# Why Use Nested Classes?

- It is a way of logically grouping classes that are only used in one place.
- It increases encapsulation.
- It can lead to more readable and maintainable code.

# Note:

- A nested class is a member of its outer class. Non-static nested classes (**inner classes**) have access to other members of the outer class, even if they are declared private.
- Static nested classes do not have access to other members of the enclosing class. As a member of the OuterClass, a nested class can be declared private, public, protected, or package private.

# Example,

```java
package test;

public class OuterClass {

    String outerField = "Outer field";
    static String staticOuterField = "Static outer field";

    public class InnerClass {
        void accessMembers() {
            System.out.println(outerField);
            System.out.println(staticOuterField);
        }
    }

    public static class StaticNestedClass {
        void accessMembers(OuterClass outer) {
            // Compiler error: Cannot make a static reference to the non-static
            //     field outerField
            // System.out.println(outerField);
            System.out.println(outer.outerField);
            System.out.println(staticOuterField);
        }
    }

    public static void main(String[] args) {
        System.out.println("Inner class:");
        System.out.println("------------");
        OuterClass outerObject = new OuterClass();
        OuterClass.InnerClass innerObject = outerObject.new InnerClass();
        innerObject.accessMembers();

        System.out.println("\nStatic nested class:");
        System.out.println("--------------------");
        StaticNestedClass staticNestedObject = new StaticNestedClass();
        staticNestedObject.accessMembers(outerObject);
    }
}
```

# Interface

Software objects also communicate via interfaces. A Java interface describes a set of methods that can be called on an object to tell it, for example, to perform some tasks or return some piece of information. An interface declaration begins with the keyword **interface** and contains only constants and abstract methods.

All methods declared in an interface are implicitly **public abstract** methods, and all fields are implicitly **public, static and final**.

To use an interface, a concrete class must specify that it **implements** the interface and must declare each method in the interface with the signature specified in the interface declaration.

# Example,

```
interface Bicycle {

    void changeCadence(int newValue);

    void changeGear(int newValue);

    void speedUp(int increment);

    void applyBrakes(int decrement);
}

class ACMEBicycle implements Bicycle {

    int cadence = 0;
    int speed = 0;
    int gear = 1;

    void changeCadence(int newValue) {
         cadence = newValue;
    }

    void changeGear(int newValue) {
         gear = newValue;
    }

    void speedUp(int increment) {
         speed = speed + increment;
    }

    void applyBrakes(int decrement) {
         speed = speed - decrement;
    }

    void printStates() {
         System.out.println("cadence:" + cadence + " speed:" + speed
                            + " gear:" + gear);
    }
}
```

# Homework>>

- Why would we use an interface?
- Explain with example: An interface can extend multiple interfaces.
  A class can implement multiple interfaces.

# ArrayList

The collection class **ArrayList<T>** (from package java.util) provides a convenient solution to store sequences of objects. ArrayList automatically change their size at execution time to accommodate additional elements. Only nonprimitive types can be used with ArrayList. The following are some common methods of ArrayList:

| Method | Description |
|---|---|
| `add` | Adds an element to the end of the `ArrayList`. |
| `clear` | Removes all the elements from the `ArrayList`. |
| `contains` | Returns `true` if the `ArrayList` contains the specified element; otherwise, returns `false`. |
| `get` | Returns the element at the specified index. |
| `indexOf` | Returns the index of the first occurrence of the specified element in the `ArrayList`. |
| `remove` | Overloaded. Removes the first occurrence of the specified value or the element at the specified index. |
| `size` | Returns the number of elements stored in the `ArrayList`. |
| `trimToSize` | Trims the capacity of the `ArrayList` to current number of elements. |

# Example,

```
3  import java.util.ArrayList;
4
5  public class ArrayListExample {
6      public static void main( String[] args ){
7          // create a new ArrayList of Strings with an initial capacity of 10
8          ArrayList< String > items = new ArrayList< String >();
9          items.add( "red" ); // append an item to the list
10         items.add( 0, "yellow" ); // insert the value at index 0
11
12         // header
13         System.out.print("Display list contents with counter-controlled loop:" );
14
15         // display the colors in the list
16         for ( int i = 0; i < items.size(); i++ )
17             System.out.printf( " %s", items.get( i ) );
18
19         // display colors using foreach in the display method
20         display( items, "\nDisplay list contents with enhanced for statement:" );
21
22         items.add( "green" ); // add "green" to the end of the list
23         items.add( "yellow" ); // add "yellow" to the end of the list
24
25         display( items, "List with two new elements:" );
26
27          items.remove( "yellow" ); // remove the first "yellow"
28          display( items, "Remove first instance of yellow:" );
29
30          items.remove( 1 ); // remove item at index 1
31          display( items, "Remove second list element (green):" );
32
33          // check if a value is in the List
34          System.out.printf( "\"red\" is %sin the list\n",
35          items.contains( "red" ) ? "": "not " );
36
37          // display number of elements in the List
38          System.out.printf( "Size: %s\n", items.size() );
39     } // end main
40
41     // display the ArrayList's elements on the console
42     public static void display( ArrayList< String > items, String header ) {
43
44         System.out.print( header ); // display header
45         // display each element in items
46         for ( String item : items )
47             System.out.printf( " %s", item );
48         System.out.println(); // display end of line
49     } // end method display
50 } // end class ArrayListCollection
```